

EXERCISE 02 · SQL FOUNDATIONS

MOSHE MATABANE

```
--EXERCISE 2

--01: THE STUDENTS TABLE
=====

--Step 1: Create a catalog
CREATE CATALOG IF NOT EXISTS moshe;

--Step 2: Create schema
CREATE SCHEMA IF NOT EXISTS moshe.school;

--Step 3: Create table
--CREATE TABLE IF NOT EXISTS moshe.school.students (student_id INT,
name STRING,
age INT,
department STRING);
```

```
--Question 1: List all distinct departments in the students table
SELECT DISTINCT department
FROM moshe.school.students;
```

Table	+	Q	F	I	Q	S	^
A ^B _C department							
1	IT						
2	HR						
3	Finance						

```
--Question 2: Get the average age of students per department
SELECT department,
AVG(age) AS avg_age
FROM moshe.school.students
GROUP BY department;
```

Table	+	Q	F	I	Q	S	^
A ^B _C department	1.2 avg_age						
1	IT	20.5					
2	HR	22					
3	Finance	23					

```
--Question 3: Show departments with more than 1 student
SELECT department, COUNT(*) AS student_count
FROM moshe.school.students
GROUP BY department
HAVING student_count > 1;
```

	A ^B _C department	1 ² ₃ student_count
1	IT	2
2	HR	2

```
--Question 4: Get all students whose age is between 21 and 23
SELECT student_id,
name,
age,
department
FROM moshe.school.students
WHERE age BETWEEN 21 AND 23;
```

	1 ² ₃ student_id	A ^B _C name	1 ² ₃ age	A ^B _C department
1	2	Bob	22	HR
2	3	Charlie	21	IT
3	4	Diana	23	Finance
4	5	Eve	22	HR

```
--Question 5: List all students in the IT or HR who are older than 21
SELECT student_id,
name,
age,
department
FROM moshe.school.students
WHERE department IN ('IT', 'HR') AND age > 21;
```

	1 ² ₃ student_id	A ^B _C name	1 ² ₃ age	A ^B _C department
1	2	Bob	22	HR
2	5	Eve	22	HR

```
--02: THE COURSES TABLE
=====

--Step 1: Create table
CREATE TABLE IF NOT EXISTS moshe.school.courses (course_id INT, course_name STRING,
department STRING,
credits INT);

--Step 2: Insert data into the table
INSERT INTO moshe.school.courses VALUES
(101, 'SQL Basics', 'IT', 3),
(102, 'Python', 'IT', 4),
(103, 'Data Science', 'IT', 4),
(104, 'Excel', 'Finance', 2),
(105, 'Statistics', 'HR', 3);

--Step 3: Check if the table is correct
SELECT*
FROM moshe.school.courses;
```

```
--Question 6: Show total credits per department, only for departments with more than 5 total credits.
SELECT department, SUM(credits) AS total_credits
FROM moshe.school.courses
GROUP BY department
HAVING total_credits > 5;
```

	department	total_credits
1	IT	22
2	HR	6

```
--Question 7: List all courses that do not have 4 credits.
SELECT*
FROM moshe.school.courses
WHERE credits != 4;
```

	course_id	course_name	department	credits
1	101	SQL Basics	IT	3
2	104	Excel	Finance	2
3	105	Statistics	HR	3
4	101	SQL Basics	IT	3
5	104	Excel	Finance	2
6	105	Statistics	HR	3

--Question 8: Show the top 3 courses by credits in descending order.

```
SELECT course_id, course_name, credits
FROM moshe.school.courses
ORDER BY credits DESC
LIMIT 3;
```

	course_id	course_name	credits
1	102	Python	4
2	103	Data Science	4
3	102	Python	4

```
--03 : THE ENROLLMENTS TABLE
=====

--Step 1: Create table
CREATE TABLE IF NOT EXISTS moshe.school.enrollments
(enrollment_id INT,
student_id INT,
course_id INT,
grade INT);

--Step 2: Insert into the table
INSERT INTO moshe.school.enrollments VALUES
(1, 1, 101, 85),
(2, 2, 102, 78),
(3, 3, 103, 90),
(4, 4, 104, 88),
(5, 5, 105, 82);

--Step 3: Check if the table is correct
SELECT*
FROM moshe.school.enrollments
```

--Question 9: Get the maximum, minimum, and average grade across all enrollment.

```
SELECT MIN(grade) AS min_grade, MAX(grade) AS max_grade, AVG(grade) AS avg_grade
FROM moshe.school.enrollments;
```

	min_grade	max_grade	avg_grade
1	78	90	84.6

```
--Question 10: Count how many enrollments exist per course.
SELECT course_id, Count(*) AS enrollment_count
FROM moshe.school.enrollments
GROUP BY course_id
```

	1 ² ₃ course_id	1 ² ₃ enrollment_count
1	101	1
2	102	1
3	103	1
4	104	1
5	105	1

```
--04: THE SALARIES TABLE
=====

--Step 1: Create table
CREATE TABLE IF NOT EXISTS moshe.school.salaries
(employee_id INT,
name STRING,
department STRING,
salary INT,
bonus INT);

--Step 2: Insert into the table.
INSERT INTO moshe.school.salaries VALUES
(1, 'Tom', 'IT', 60000, 5000),
(2, 'Jerry', 'HR', 55000, 4000),
(3, 'Spike', 'Finance', 70000, 6000),
(4, 'Tyke', 'IT', 62000, 5500),
(5, 'Butch', 'HR', 54000, 3500);

--Step 3: Check if the table is correct.
SELECT*
FROM moshe.school.salaries;
```

```
--Question 11: Find total salary and total bonus per department.
SELECT department, SUM(salary) AS total_salary, SUM(bonus) AS total_bonus
FROM moshe.school.salaries
GROUP BY department;
```

	A ^B _C department	1 ² ₃ total_salary	1 ² ₃ total_bonus
1	IT	122000	10500
2	HR	109000	7500
3	Finance	70000	6000

```
--Question 12: Show departments where avarage salary is above 55,000
```

```
SELECT department, AVG(salary) AS avg_salary
```

```
FROM moshe.school.salaries
```

GROUP BY department

```
HAVING avg_salary > 55000
```

	A ^B C department	1.2 avg_salary
1	IT	61000
2	Finance	70000

```
--Question 13: List employees whose salary plus bonus is greater than 60000
```

```
SELECT employee id, name, salary, bonus, SUM(salary + bonus) AS total compensation
```

```
FROM moshe.school.salaries
```

```
GROUP BY employee id, name, salary, bonus
```

```
HAVING total_compensation > 60000
```

Table						
	1 ² ₃ employee_id	A ^B _C name	1 ² ₃ salary	1 ² ₃ bonus	1 ² ₃ total_compensation	
1	1	Tom	60000	5000	65000	
2	3	Spike	70000	6000	76000	
3	4	Tyke	62000	5500	67500	

```
--05: THE PROJECT TABLE
=====

--Step 1: Create table
CREATE TABLE IF NOT EXISTS moshe.school.projects
(project_id INT,
project_name STRING,
department STRING,
budget INT);

--Step 2: Instert into table
INSERT INTO moshe.school.projects VALUES
(1, 'AI App', 'IT', 120000),
(2, 'Payroll System', 'Finance', 80000),
(3, 'Dashboard', 'IT', 150000),
(4, 'Website', 'Marketing', 60000),
(5, 'HR Portal', 'HR', 50000);

--Step 3: Check if the table is correct.
SELECT *
FROM moshe.school.projects;
```

```
--Question 14: Show total and avarage budget per department. Only include departments with avarage budget above 70000
SELECT department, SUM(budget) AS total_budget, AVG(budget) AS avg_budget
FROM moshe.school.projects
GROUP BY department
HAVING avg_budget > 70000;
```

Table			
	department	total_budget	avg_budget
1	IT	270000	135000
2	Finance	80000	80000

```
--Question 15: List all projects with budgets between 50000 and 120000, excluding the marketing department.
SELECT*
FROM moshe.school.projects
WHERE budget BETWEEN 50000 AND 120000 AND department != 'Marketing';
```

Table				
	project_id	project_name	department	budget
1	1	AI App	IT	120000
2	2	Payroll System	Finance	80000
3	5	HR Portal	HR	50000